

Programmation C

9 – Programmation modulaire

Alain CROUZIL, Emmanuel LAVINAL

`alain.crouzil@irit.fr, emmanuel.lavinal@irit.fr`

Département d'Informatique
Faculté Sciences et Ingénierie
Université de Toulouse

Licence
Semestre pair 2025-2026

 | Département
Informatique

 Faculté
sciences et
ingénierie
Université
de Toulouse

Sommaire

- 1 Introduction
- 2 Modules
- 3 Compilation séparée
- 4 Bibliothèques sous Unix avec `gcc`

Sommaire

- 1 Introduction
- 2 Modules
- 3 Compilation séparée
- 4 Bibliothèques sous Unix avec `gcc`

Introduction (1/2)

Objectifs

- **Lisibilité** : pour comprendre le comportement et la mise en œuvre d'un programme en lisant le code source.
- **Maintenabilité** : pour détecter des erreurs dans une partie de programme et les corriger sans en provoquer de nouvelles dans le reste du programme.
- **Portabilité** : pour adapter les fonctionnalités d'un programme à un nouvel environnement.
- **Extensibilité** : pour modifier l'implémentation d'une partie de programme ou rajouter des fonctionnalités sans modifier le comportement de l'ensemble.
- **Réutilisabilité** : pour utiliser des parties de programmes différents pour en écrire de nouveaux.

Introduction (2/2)

Principes

- **Abstraction des constantes** : utiliser des constantes symboliques.

Exemple :

```
...  
int tab[10];  
...  
for (int i=0; i<10; i++)  
...  
→  
#define TAILLE 10  
...  
int tab[TAILLE];  
...  
for (int i=0; i<TAILLE; i++)  
...
```

- **Factorisation de code** : éviter la duplication de code en définissant des fonctions.
- **Masquage de l'implémentation** : séparer, pour chaque module, son *interface* (ce qui doit être connu par un utilisateur du module) de son *implémentation* (la mise en œuvre).
 - On doit pouvoir utiliser le module sans en connaître l'implémentation.
 - On doit pouvoir modifier l'implémentation sans que l'utilisateur en soit affecté.

Sommaire

- 1 Introduction
- 2 Modules
- 3 Compilation séparée
- 4 Bibliothèques sous Unix avec `gcc`

Constitution d'un module (1/2)

Fichier d'en-tête

Fichier d'extension .h constituant son interface et contenant principalement :

- des définitions de structures (**struct...**) ;
- des définitions de synonymes de types (**typedef...**) ;
- des définitions de constantes symboliques (**#define...**) ;
- des déclarations de fonctions externes (**extern...**) ;
- des inclusions de fichiers d'en-tête (**#include...**) ;
- des commentaires.

Constitution d'un module (2/2)

Fichier d'implémentation

Fichier d'extension .c contenant :

- l'inclusion du fichier d'en-tête correspondant ;
- des inclusions de fichiers d'en-tête ;
- des définitions de structures et de synonymes de types privés ;
- des définitions de constantes symboliques privées ;
- des définitions de fonctions privées, c'est-à-dire locales au fichier (**static...**) ;
- les définitions des fonctions déclarées dans le fichier d'en-tête correspondant.

Utilisation d'un module

L'utilisateur d'un module doit :

- inclure le fichier d'en-tête ;
- prendre en compte le module lors de l'édition de liens.

Encapsulation des traitements

La partie visible du module, c'est-à-dire le fichier d'en-tête, ne contient que les déclarations des fonctions (les références aux fonctions) et pas leur définitions. Leurs corps sont « cachés » dans le fichier d'implémentation.

module.h

```
/* PARTIE PUBLIQUE */  
...  
/* Commentaire */  
extern ...f(...);  
...
```

module.c

```
/* PARTIE PRIVÉE */  
...  
... f(...)  
{  
    ...  
    ...  
    ...  
}  
...
```

Encapsulation des données (1/2)

Objectifs

- Il n'est pas possible d'agir directement sur les données d'un module.
- Il est nécessaire de passer par l'appel des fonctions publiques du module (*méthodes* en programmation orientée objet) qui constituent une interface obligatoire.

Abstraction des données

Vu de l'extérieur, un module se caractérise uniquement par les spécifications de ses fonctions publiques (contenues dans le fichier d'en-tête), les détails d'implémentation étant sans importance.

⇒ facilite la *maintenance* :

- une modification éventuelle de la structure des données d'un module n'a d'incidence que sur le module lui-même ;
- les utilisateurs du module ne seront pas concernés par la teneur de cette modification.

Encapsulation des données (2/2)

Pour le module

Il est possible d'utiliser la *définition retardée des structures* :

- déclarer un synonyme de type « pointeur vers une structure » dans le fichier d'en-tête ;
- définir le contenu (les champs) de la structure uniquement dans le fichier d'implémentation.

Le concepteur du module peut :

- accéder au contenu de la structure pointée dans le fichier d'implémentation ;
- si nécessaire, changer d'implémentation sans changer le contenu du fichier d'en-tête, c'est-à-dire de manière transparente pour l'utilisateur du module.

Seule contrainte : l'implémentation doit passer par une structure.

Pour l'utilisateur du module

- Il peut déclarer des variables de ce type.
- Il peut les manipuler : les faire passer en paramètre de fonctions par exemple.
- Il ne peut pas accéder à l'objet pointé (au contenu de la structure pointée).

Exemple

module.h

```
...  
typedef struct sListe *tListe;  
...  
extern tListe CreerListe(void);  
...
```

module.c

```
...  
struct sListe  
{  
    int x;  
    int y;  
    struct sListe *pSuivant;  
    /* ou tListe pSuivant; */  
};  
...  
tListe CreerListe(void)  
{  
    return NULL;  
}  
...
```

Exemple

module.h

```
...
typedef struct sListe *tListe;
...
extern tListe CreerListe(void);
...
```

module.c

```
...
#define MAX 1000
struct sPoint { int x,y; };
struct sListe
{
    int NbPoints;
    struct sPoint TabPoints[MAX];
};
...
tListe CreerListe(void)
{
    tListe lst;
    lst=malloc(sizeof(struct sListe));
    lst->NbPoints=0;
    return lst;
}
...
```

Le concepteur du module peut alors changer l'implémentation de la liste de points en modifiant uniquement le fichier module.c.

Inclusions multiples

Situation

- Pour pouvoir utiliser un module, il faut inclure son fichier d'en-tête.
- Des modules peuvent utiliser d'autres modules.
- Si un même fichier d'en-tête a été inclus plusieurs fois, le compilateur peut détecter des erreurs comme la redéfinition de structure.

Utilisation du préprocesseur

module.h

```
#ifndef MODULE_H
#define MODULE_H
/* Contenu du fichier d'en-tête */
...
...
#endif /*MODULE_H*/
```

- Lors de la première inclusion : la constante symbolique `MODULE_H` n'est pas définie ; elle le devient et le contenu du fichier d'en-tête est inclus.
- À partir de la deuxième inclusion, la constante symbolique `MODULE_H` étant définie, le contenu du fichier d'en-tête n'est plus inclus.

Sommaire

- 1 Introduction
- 2 Modules
- 3 Compilation séparée**
- 4 Bibliothèques sous Unix avec `gcc`

Compilation d'un module

Production d'un fichier objet

- Pas de fonction `main` dans un module $\Rightarrow$ pas de production d'un fichier exécutable
- Mais production d'un fichier objet (compilation sans édition de lien) : option `-c` de `gcc`

Exemple : `gcc -c module.c` produit le fichier objet `module.o`

Compilation d'un programme (1/6)

Exemple

Fichier `prog.c` contenant la fonction `main` et utilisant les modules (`module1.h`, `module1.c`) et (`module2.h`, `module2.c`) :

prog.c

```
...  
#include "module1.h"  
#include "module2.h"  
...  
int main(void)  
...
```

module1.c

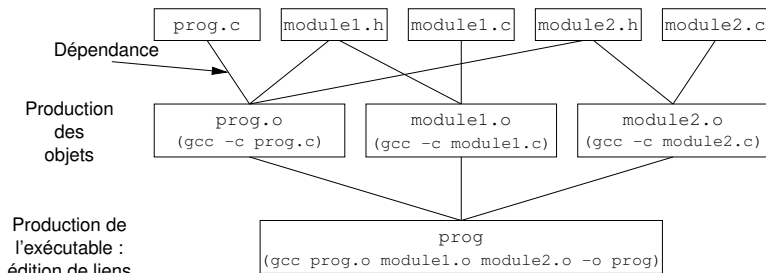
```
...  
#include "module1.h"  
...
```

module2.c

```
...  
#include "module2.h"  
...
```

Compilation d'un programme (2/6)

Graphe de dépendance



- Si le fichier `module1.c` est modifié, il suffit de relancer les deux commandes suivantes :

```
gcc -c module1.c
```

```
gcc prog.o module1.o module2.o -o prog
```
- Pour gérer ces dépendances et ne recompiler que ce qui doit l'être, on peut faire appel à l'utilitaire `make`.

Compilation d'un programme (3/6)

Utilitaire `make` (Unix)

● Invocation :

- si le fichier de description des dépendances est dans le répertoire courant et s'appelle `makefile` ou `Makefile` :

```
make
```

- si le fichier `fichier_des_dépendances` est le fichier de description des dépendances :

```
make -f fichier_des_dépendances
```

● Syntaxe d'une description de dépendances :

```
cible:liste_de_fichiers
```

```
↳commande
```

- `↳` représente le caractère tabulation (attention aux éditeurs de texte qui transforment automatiquement les tabulations en espaces).
- Cette description signifie : « *cible* dépend des fichiers de la liste ». Si au moins un des fichiers de la liste a été modifié plus récemment que la *cible*, alors la *commande* est exécutée.

- **Ordre des descriptions de dépendances** : la première cible doit être l'objectif final.

Compilation d'un programme (4/6)

Utilitaire `make` (suite)

- **Commentaires :** *# ceci est un commentaire*
- **Variables :**
 - Affectation : *identificateur=valeur* (par exemple : `CC=gcc`)
 - Utilisation : *\$(identificateur)* (par exemple : `$(CC)`)

Compilation d'un programme (5/6)

Fichier `Makefile` de description des dépendances du programme `prog`

```
CC=gcc
CFLAGS=-std=c11 -Wpedantic -Wall -Wextra -Werror=vla
LDLIBS=-lm
RM=rm -f
OBJ=prog.o module1.o module2.o
EXE=prog

# Pour produire le fichier exécutable
$(EXE):$(OBJ)
    $(CC) $(CFLAGS) $(OBJ) -o $(EXE) $(LDLIBS)

# Pour produire les fichiers objets
prog.o:prog.c module1.h module2.h
    $(CC) $(CFLAGS) -c prog.c
module1.o:module1.c module1.h
    $(CC) $(CFLAGS) -c module1.c
module2.o:module2.c module2.h
    $(CC) $(CFLAGS) -c module2.c

# Pour effacer les objets et les exécutables
clean:
    $(RM) $(OBJ) $(EXE)
```

Compilation d'un programme (6/6)

Commandes

- Pour produire l'exécutable `prog` :
`make`
- Pour supprimer les fichiers objets et le fichier exécutable :
`make clean`

Sommaire

- 1 Introduction
- 2 Modules
- 3 Compilation séparée
- 4 Bibliothèques sous Unix avec gcc

Bibliothèques sous Unix avec gcc (1/4)

De quoi s'agit-il ?

Une bibliothèque :

- consiste en une archive qui réunit un ensemble de fichiers objets (d'extension `.o`) en un seul fichier ;
- correspond à un module qui a été découpé en « sous-modules » :
 - chaque sous-module est un fichier source (d'extension `.c`) avec, éventuellement, un fichier d'en-tête associé (d'extension `.h`) ;
 - il est aussi possible d'avoir un fichier d'en-tête global associé à la bibliothèque.

Bibliothèques sous Unix avec gcc (2/4)

Bibliothèques statiques

Fichiers d'extension `.a` qui sont inclus dans le fichier exécutable au moment de l'édition de liens.

Exemple

- **Création** : `ar -ru libbiblio.a fich1.o fich2.o`
 - `-r` : les objets existant dans l'archive doivent être remplacés ;
 - `-u` : n'insère dans l'archive que les objets plus récents que les objets existants de même nom.
- **Utilisation** : `gcc exe.o -lbiblio -o exe`
 - `-l` : réalise l'édition de liens avec la bibliothèque `libbiblio.a`, sachant que le préfixe `lib` et l'extension sont rajoutés automatiquement ;
 - si la bibliothèque n'est pas dans le répertoire courant, il est possible d'en préciser le chemin d'accès avec l'option `-L` suivie de la désignation du répertoire.

Bibliothèques sous Unix avec gcc (3/4)

Bibliothèques dynamiques partagées

Fichiers d'extension `.so` qui ne sont pas inclus dans le fichier exécutable au moment de l'édition de liens mais qui sont liés seulement au moment de l'exécution.

- **Avantages :**

- les exécutables sont moins volumineux ;
- on peut modifier la bibliothèque sans devoir refaire une édition de liens ;
- un seul exemplaire du code de la bibliothèque est chargé en mémoire même si plusieurs processus l'utilisent.

- **Inconvénients :**

- les exécutables ne sont pas « auto-suffisants » ;
- un exécutable peut changer de comportement si la bibliothèque est modifiée ;
- le chemin d'accès au répertoire contenant la bibliothèque doit se trouver dans la variable d'environnement `LD_LIBRARY_PATH`.

Bibliothèques sous Unix avec gcc (4/4)

Bibliothèques dynamiques partagées (suite)

Exemple

- **Création** : `gcc -shared -o libbiblio.so fich1.o fich2.o`
- **Utilisation** : `gcc exe.o -lbiblio -o exe`
- **Exécution** :
 - Rajouter à la liste des répertoires contenus dans la variable d'environnement `LD_LIBRARY_PATH` le répertoire contenant la bibliothèque.
 - Par exemple, avec l'interpréteur de commandes `bash`, on peut rajouter dans le fichier `.bash_profile` les lignes suivantes :

```
LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/home/moi/chemin_bibliotheque  
export LD_LIBRARY_PATH
```

- **Vérification des dépendances dynamiques** :
 - Linux : `ldd exe`
 - macOS : `otool -L exe`